```
Start coding or generate with AI.
```

ILSTU Campus Services Prototype

```
# a) Markdown heading is in the cell above

# Exercise 1, Question 1(b)
print("=== ILSTU Campus Services Decision-Support Tool ===")
print("This program helps students calculate café bills, check event ticket affordability,")
print("and evaluate campus service decisions before spending money.\n")

# Exercise 1, Question 1(c)
student_name = "Alex Johnson"
student_id = "Cor012345"  # stored as text (string)
current_semester = 4
is_enrolled = True

print("Student Name:", student_name)
print("Student ID:", student_id)
print("Current Semester:", current_semester)
print("Currently Enrolled:", is_enrolled)
print("-" * 40)

# Exercise 1, Question 1(d)
print("Data type of student_name:", type(student_name))
print("Data type of student_id:", type(student_id))
print("Data type of current_semester:", type(current_semester))
print("Data type of is_enrolled:", type(is_enrolled))
```

```
=== ILSTU Campus Services Decision-Support Tool ===
This program helps students calculate café bills, check event ticket affordability,
and evaluate campus service decisions before spending money.

Student Name: Alex Johnson
Student ID: Cor012345
Current Semester: 4
Currently Enrolled: True
----------------------------------------
Data type of student_name: <class 'str'>
Data type of student_id: <class 'str'>
Data type of current_semester: <class 'int'>
Data type of is_enrolled: <class 'bool'>
```

```
Start coding or generate with AI.
```

I stored the student ID as text because it is just a label and I never do calculations with it. If it were an integer, Python would remove any zeros at the start. My ID also has letters in it, and letters can't be stored in an integer.

```
Start coding or generate with AI.
```

```
# Exercise 1, Question 2(a)
meal_price = 850
drink_price = 180
subtotal = meal_price + drink_price
print("Subtotal: USD", subtotal)

# Exercise 1, Question 2(b)
discount_amount = meal_price * 0.10
discounted_meal_price = meal_price - discount_amount
final_bill = discounted_meal_price + drink_price
print("Discount amount: USD", discount_amount)
print("Discounted meal price: USD", discounted_meal_price)
print("Final bill: USD", final_bill)

# Exercise 1, Question 2(c)
print("Final bill (2 decimal places): USD", round(final_bill, 2))

# Exercise 1, Question 2(d)
# Expression used: final_bill = (meal_price - meal_price * 0.10) + drink_price
```

```
# Comparison: 10% applied to the whole subtotal (wrong method, for explanation only)
wrong_discount = subtotal * 0.10
wrong_final_bill = subtotal - wrong_discount
print("Wrong method discount: USD", wrong_discount)
print("Wrong method final bill: USD", wrong_final_bill)
```

```
Subtotal: USD 1030
Discount amount: USD 85.0
Discounted meal price: USD 765.0
Final bill: USD 945.0
Final bill (2 decimal places): USD 945.0
Wrong method discount: USD 103.0
Wrong method final bill: USD 927.0
```

If I took 10% off the whole subtotal, the drink would also get a discount. That would give a discount of USD 103 and a final bill of USD 927. The correct way gives a discount of USD 85 and a final bill of USD 945. The rule says only the meal is discounted, so the drink has to stay at full price.

```
# Exercise 1, Question 3(a)
available_seats = 18
requested_seats = 4
ticket_price = 650
student_budget = 2500

# Exercise 1, Question 3(b)
total_cost = requested_seats * ticket_price
seats_available = requested_seats <= available_seats
within_budget = total_cost <= student_budget
both_true = seats_available and within_budget

# Exercise 1, Question 3(c)
print("Total ticket cost: USD", total_cost)
print("The statement 'the requested seats are available' is", seats_available)
print("The statement 'the total cost is within the budget' is", within_budget)
print("The statement 'both conditions are true' is", both_true)

# Exercise 1, Question 3(d)
requested_seats = 5
student_budget = 3000

total_cost = requested_seats * ticket_price
seats_available = requested_seats <= available_seats
within_budget = total_cost <= student_budget
both_true = seats_available and within_budget

print("--- After changing requested_seats to 5 and student_budget to 3000 ---")
print("Total ticket cost: USD", total_cost)
print("The statement 'the requested seats are available' is", seats_available)
print("The statement 'the total cost is within the budget' is", within_budget)
print("The statement 'both conditions are true' is", both_true)
```

```
Total ticket cost: USD 2600
The statement 'the requested seats are available' is True
The statement 'the total cost is within the budget' is False
The statement 'both conditions are true' is False
--- After changing requested_seats to 5 and student_budget to 3000 ---
Total ticket cost: USD 3250
The statement 'the requested seats are available' is True
The statement 'the total cost is within the budget' is False
The statement 'both conditions are true' is False
```

None of the results changed. The seats are still available because 5 is less than 18. The budget check is still False: the cost went from 2600 to 3250, while the budget only went from 2500 to 3000. The cost grew by 650 and the budget grew by 500, so the total is still over budget. Since one condition is False, the "and" result stays False.

```
Start coding or generate with AI.
```

```
# Exercise 1, Question 4(d) - Test 1: 2 tickets, budget 1500
# Exercise 1, Question 4(a)
student_name = input("Enter your name: ")
ticket_count_text = input("Enter the number of tickets you want to buy: ")

# Exercise 1, Question 4(b)
```

```
ticket_count = int(ticket_count_text)
ticket_price = 650
total_cost = ticket_count * ticket_price
print(student_name, "- total cost for", ticket_count, "tickets: USD", total_cost)

# Exercise 1, Question 4(c)
budget = float(input("Enter your available budget in USD: "))
is_affordable = total_cost <= budget
print("Purchase is affordable:", is_affordable)
```

```
Enter your name: Alex
Enter the number of tickets you want to buy: 2
Alex - total cost for 2 tickets: USD 1300
Enter your available budget in USD: 1500
Purchase is affordable: True
```

```
Start coding or generate with AI.
```

```
# Exercise 1, Question 4(d) - Test 2: 3 tickets, budget 2000
# Exercise 1, Question 4(a)
student_name = input("Enter your name: ")
ticket_count_text = input("Enter the number of tickets you want to buy: ")

# Exercise 1, Question 4(b)
ticket_count = int(ticket_count_text)
ticket_price = 650
total_cost = ticket_count * ticket_price
print(student_name, "- total cost for", ticket_count, "tickets: USD", total_cost)

# Exercise 1, Question 4(c)
budget = float(input("Enter your available budget in USD: "))
is_affordable = total_cost <= budget
print("Purchase is affordable:", is_affordable)
```

```
Enter your name: Alex
Enter the number of tickets you want to buy: 3
Alex - total cost for 3 tickets: USD 1950
Enter your available budget in USD: 2000
Purchase is affordable: True
```

Test 1: 2 tickets, budget USD 1500 -> total USD 1300 -> affordable: True

Test 2: 3 tickets, budget USD 2000 -> total USD 1950 -> affordable: True

```
Start coding or generate with AI.
```

Problem 1: `print_jobs = "3"` is text (a string), not a number. So `print_jobs * price_per_job` repeats the text "3" 45 times instead of multiplying, giving a wrong total (a logic error).

Problem 2: `"Total cost: USD " + total` joins text and `total` directly. Once `print_jobs` is fixed to a number, `total` becomes an integer, and Python can't add text and an integer, so it raises a TypeError. The number must be converted with `str()`.

```
Start coding or generate with AI.
```

```
# Exercise 1, Question 5(b)
print("ILSTU Printing Desk")
print_jobs = 3
price_per_job = 45
total = print_jobs * price_per_job
print("Total cost: USD " + str(total))
```

```
ILSTU Printing Desk
Total cost: USD 135
```

```
Start coding or generate with AI.
```

Syntax error: the code breaks Python's grammar rules, so it cannot run at all. Example: `print("Total cost"` with a missing closing parenthesis.

Runtime error: the code is written correctly but crashes while running. Example: `"Total cost: USD " + 135` raises a TypeError because text and a number cannot be added.

Logic error: the code runs without crashing but gives a wrong answer. Example in this billing scenario: writing `total = print_jobs + price_per_job` gives 48 instead of the correct 135.

Exercise 2

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 1(a)-(c)

# Combination 1: enrolled, completed module
is_enrolled = True
completed_module = True
is_eligible = is_enrolled and completed_module
print("Enrolled:", is_enrolled, "| Module completed:", completed_module)
if is_eligible:
    print("Registration is permitted.")
else:
    print("Registration is not permitted.")
print("-" * 40)

# Combination 2: enrolled, module not completed
is_enrolled = True
completed_module = False
is_eligible = is_enrolled and completed_module
print("Enrolled:", is_enrolled, "| Module completed:", completed_module)
if is_eligible:
    print("Registration is permitted.")
else:
    print("Registration is not permitted.")
print("-" * 40)

# Combination 3: not enrolled, module completed
is_enrolled = False
completed_module = True
is_eligible = is_enrolled and completed_module
print("Enrolled:", is_enrolled, "| Module completed:", completed_module)
if is_eligible:
    print("Registration is permitted.")
else:
    print("Registration is not permitted.")
print("-" * 40)

# Combination 4: not enrolled, module not completed
is_enrolled = False
completed_module = False
is_eligible = is_enrolled and completed_module
print("Enrolled:", is_enrolled, "| Module completed:", completed_module)
if is_eligible:
    print("Registration is permitted.")
else:
    print("Registration is not permitted.")
```

```
Enrolled: True | Module completed: True
Registration is permitted.
----------------------------------------
Enrolled: True | Module completed: False
Registration is not permitted.
----------------------------------------
Enrolled: False | Module completed: True
Registration is not permitted.
----------------------------------------
Enrolled: False | Module completed: False
Registration is not permitted.
```

Only the first combination, where the student is enrolled and has finished the module, allows registration. Both conditions have to be true because the rule uses AND. A student needs to be currently enrolled and also have completed the introductory programming module. If

either one is False, the whole expression is False, so the student cannot register.

```
Start coding or generate with AI.
```

Exercise 2, Question 2

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 2 - Record C101
# a) Store the values in variables
meal_price = 850
drink_price = 180
card_valid = True
meal_plan = False

# b) Apply the correct rule (meal plan is checked first)
if meal_plan:
    meal_charge = 0
elif card_valid:
    meal_charge = meal_price - meal_price * 0.10
else:
    meal_charge = meal_price

final_bill = meal_charge + drink_price

# c) Print the results
print("Order C101")
print("Meal charge after benefits: USD", meal_charge)
print("Drink charge: USD", drink_price)
print("Final bill: USD", final_bill)
```

```
Order C101
Meal charge after benefits: USD 765.0
Drink charge: USD 180
Final bill: USD 945.0
```

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 2 - Record C102
meal_price = 620
drink_price = 150
card_valid = True
meal_plan = True

if meal_plan:
    meal_charge = 0
elif card_valid:
    meal_charge = meal_price - meal_price * 0.10
else:
    meal_charge = meal_price

final_bill = meal_charge + drink_price

print("Order C102")
print("Meal charge after benefits: USD", meal_charge)
print("Drink charge: USD", drink_price)
print("Final bill: USD", final_bill)
```

```
Order C102
Meal charge after benefits: USD 0
Drink charge: USD 150
Final bill: USD 150
```

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 2 - Record C104
meal_price = 780
drink_price = 0
card_valid = False
meal_plan = False

if meal_plan:
```

```
    meal_charge = 0
elif card_valid:
    meal_charge = meal_price - meal_price * 0.10
else:
    meal_charge = meal_price

final_bill = meal_charge + drink_price

print("Order C104")
print("Meal charge after benefits: USD", meal_charge)
print("Drink charge: USD", drink_price)
print("Final bill: USD", final_bill)
```

```
Order C104
Meal charge after benefits: USD 780
Drink charge: USD 0
Final bill: USD 780
```

The order of the conditions matters because Python stops at the first condition that is true. I check the meal plan first, so a student on the meal plan only gets the free meal and the card discount is skipped. If both benefits were applied to a meal-plan student, they would get two benefits when the rules allow only one. For example, if the card check came first, student C102 would pay USD 558 for the meal instead of getting it free, so the bill would be USD 708 instead of the correct USD 150.

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 3(a)
wait_time = float(input("Enter the estimated wait time in minutes: "))

# Exercise 2, Question 3(b)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
```

```
Enter the estimated wait time in minutes: 10
Short wait
```

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 3(c)
wait_time = -2
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)

wait_time = 0
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)
```

```
wait_time = 5
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)

wait_time = 6
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)

wait_time = 15
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)

wait_time = 16
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)

wait_time = 30
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
print("-" * 30)

wait_time = 31
print("Wait time:", wait_time)
if wait_time < 0:
    print("Invalid wait time")
elif wait_time <= 5:
    print("Arriving soon")
```

```
elif wait_time <= 15:
    print("Short wait")
elif wait_time <= 30:
    print("Long wait")
else:
    print("Consider another option")
```

```
Wait time: -2
Invalid wait time
------------------------------
Wait time: 0
Arriving soon
------------------------------
Wait time: 5
Arriving soon
------------------------------
Wait time: 6
Short wait
------------------------------
Wait time: 15
Short wait
------------------------------
Wait time: 16
Long wait
------------------------------
Wait time: 30
Long wait
------------------------------
Wait time: 31
Consider another option
```

The order of the conditions is important because Python stops at the first condition that is true. The negative check must come first: if it did not, -2 would match "wait_time <= 5" and be wrongly labelled "Arriving soon". Each later elif also relies on the earlier ones being false, so "wait_time <= 15" only applies to values above 5. If < were used instead of <=, the boundary value 5 would be classified incorrectly as "Short wait" instead of "Arriving soon" (15 and 30 would also be misclassified).

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 4(d) - Test 2: 10 colour pages, valid card
# Exercise 2, Question 4(a)
pages = int(input("Number of pages: "))
is_colour = input("Is the job colour? (yes/no): ").lower()
card_valid = input("Is the student card valid? (yes/no): ").lower()

# Exercise 2, Question 4(b) and 4(c)
if pages >= 1:
    # Calculate the base price
    if is_colour == "yes":
        base_price = pages * 120
    else:
        base_price = pages * 45

    # Decide whether the discount applies
    if card_valid == "yes":
        if pages >= 20:
            discount = base_price * 0.15
        else:
            discount = 0
    else:
        discount = 0

    final_price = base_price - discount
    print("Base price: USD", base_price)
    print("Discount amount: USD", discount)
    print("Final price: USD", final_price)
else:
    print("Invalid job: a job must contain at least 1 page.")
```

```
Number of pages: 10
Is the job colour? (yes/no): no
Is the student card valid? (yes/no): yes
Base price: USD 450
Discount amount: USD 0
Final price: USD 450
```

```
#Exercise 2, Question 4(d) - Test 2: 20 colour pages, valid card
# Exercise 2, Question 4(a)
pages = int(input("Number of pages: "))
is_colour = input("Is the job colour? (yes/no): ").lower()
card_valid = input("Is the student card valid? (yes/no): ").lower()

# Exercise 2, Question 4(b) and 4(c)
if pages >= 1:
    # Calculate the base price
    if is_colour == "yes":
        base_price = pages * 120
    else:
        base_price = pages * 45

    # Decide whether the discount applies
    if card_valid == "yes":
        if pages >= 20:
            discount = base_price * 0.15
        else:
            discount = 0
    else:
        discount = 0

    final_price = base_price - discount
    print("Base price: USD", base_price)
    print("Discount amount: USD", discount)
    print("Final price: USD", final_price)
else:
    print("Invalid job: a job must contain at least 1 page.")
```

```
Number of pages: 20
Is the job colour? (yes/no): yes
Is the student card valid? (yes/no): yes
Base price: USD 2400
Discount amount: USD 360.0
Final price: USD 2040.0
```

```
Start coding or generate with AI.
```

```
#Exercise 2, Question 4(d) - Test 3: 25 B&W pages, no valid card
# Exercise 2, Question 4(a)
pages = int(input("Number of pages: "))
is_colour = input("Is the job colour? (yes/no): ").lower()
card_valid = input("Is the student card valid? (yes/no): ").lower()

# Exercise 2, Question 4(b) and 4(c)
if pages >= 1:
    # Calculate the base price
    if is_colour == "yes":
        base_price = pages * 120
    else:
        base_price = pages * 45

    # Decide whether the discount applies
    if card_valid == "yes":
        if pages >= 20:
            discount = base_price * 0.15
        else:
            discount = 0
    else:
        discount = 0

    final_price = base_price - discount
    print("Base price: USD", base_price)
    print("Discount amount: USD", discount)
    print("Final price: USD", final_price)
else:
    print("Invalid job: a job must contain at least 1 page.")
```

```
Number of pages: 25
Is the job colour? (yes/no): no
Is the student card valid? (yes/no): no
Base price: USD 1125
Discount amount: USD 0
Final price: USD 1125
```

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 4(d) - Test 4: 0 pages
# Exercise 2, Question 4(a)
pages = int(input("Number of pages: "))
is_colour = input("Is the job colour? (yes/no): ").lower()
card_valid = input("Is the student card valid? (yes/no): ").lower()

# Exercise 2, Question 4(b) and 4(c)
if pages >= 1:
    # Calculate the base price
    if is_colour == "yes":
        base_price = pages * 120
    else:
        base_price = pages * 45

    # Decide whether the discount applies
    if card_valid == "yes":
        if pages >= 20:
            discount = base_price * 0.15
        else:
            discount = 0
    else:
        discount = 0

    final_price = base_price - discount
    print("Base price: USD", base_price)
    print("Discount amount: USD", discount)
    print("Final price: USD", final_price)
else:
    print("Invalid job: a job must contain at least 1 page.")
```

```
Number of pages: 0
Is the job colour? (yes/no): no
Is the student card valid? (yes/no): no
Invalid job: a job must contain at least 1 page.
```

This is an example of nested conditions because some if statements sit inside other if statements. The page count check is the outer condition, and the price and discount decisions only run inside it. Inside that, the discount check "card_valid == yes" contains another check, "pages >= 20", so the discount applies only when both are true. The condition that must be checked first is the page count (pages >= 1), because a job with zero or negative pages is invalid and should not be priced at all.

```
Start coding or generate with AI.
```

```
# Exercise 2, Question 5(a)-(c)

# Test 1: no ticket, volunteer pass
has_valid_ticket = False
has_volunteer_pass = True
is_accessibility_guest = False
print("Test 1 | Ticket:", has_valid_ticket, "| Volunteer:", has_volunteer_pass, "| Accessibility:", is_accessibility_guest)
if not has_valid_ticket:
    print("Entry denied")
elif has_volunteer_pass or is_accessibility_guest:
    print("Priority queue")
else:
    print("General entry")
print("-" * 40)

# Test 2: ticket, neither priority condition
has_valid_ticket = True
has_volunteer_pass = False
is_accessibility_guest = False
print("Test 2 | Ticket:", has_valid_ticket, "| Volunteer:", has_volunteer_pass, "| Accessibility:", is_accessibility_guest)
if not has_valid_ticket:
    print("Entry denied")
elif has_volunteer_pass or is_accessibility_guest:
    print("Priority queue")
else:
    print("General entry")
print("-" * 40)

# Test 3: ticket, volunteer pass
has_valid_ticket = True
```

```
has_volunteer_pass = True
is_accessibility_guest = False
print("Test 3 | Ticket:", has_valid_ticket, "| Volunteer:", has_volunteer_pass, "| Accessibility:", is_accessibility_guest)
if not has_valid_ticket:
    print("Entry denied")
elif has_volunteer_pass or is_accessibility_guest:
    print("Priority queue")
else:
    print("General entry")
print("-" * 40)

# Test 4: ticket, accessibility guest
has_valid_ticket = True
has_volunteer_pass = False
is_accessibility_guest = True
print("Test 4 | Ticket:", has_valid_ticket, "| Volunteer:", has_volunteer_pass, "| Accessibility:", is_accessibility_guest)
if not has_valid_ticket:
    print("Entry denied")
elif has_volunteer_pass or is_accessibility_guest:
    print("Priority queue")
else:
    print("General entry")
print("-" * 40)

# Test 5: no ticket, nothing else
has_valid_ticket = False
has_volunteer_pass = False
is_accessibility_guest = False
print("Test 5 | Ticket:", has_valid_ticket, "| Volunteer:", has_volunteer_pass, "| Accessibility:", is_accessibility_guest)
if not has_valid_ticket:
    print("Entry denied")
elif has_volunteer_pass or is_accessibility_guest:
    print("Priority queue")
else:
    print("General entry")
```

```
Test 1 | Ticket: False | Volunteer: True | Accessibility: False
Entry denied
----------------------------------------
Test 2 | Ticket: True | Volunteer: False | Accessibility: False
General entry
----------------------------------------
Test 3 | Ticket: True | Volunteer: True | Accessibility: False
Priority queue
----------------------------------------
Test 4 | Ticket: True | Volunteer: False | Accessibility: True
Priority queue
----------------------------------------
Test 5 | Ticket: False | Volunteer: False | Accessibility: False
Entry denied
```

In the priority rule, "or" means a student only needs one of the two things to get into the priority queue. If they have a ticket and either a volunteer pass or accessibility-support registration (or both), they get priority. "and" would require them to have both at the same time, so someone with only one of them would be left out of the priority queue even though the rule says they should be let in. That is why the rule uses "or".

```
Start coding or generate with AI.
```

```
# Exercise 3, Test 1: within budget, registered, valid card
# Exercise 3, Part 1 - Input and validation
budget = float(input("Available budget (USD): "))
campus_hours = float(input("Hours on campus: "))
card_valid = input("Valid student card? (yes/no): ").lower()
event_registered = input("Registered for the event? (yes/no): ").lower()

if budget < 0 or campus_hours < 0:
    print("Error: budget and campus hours cannot be negative.")
else:
    # Part 2 - Calculate the plan
    if card_valid == "yes":
        meal_cost = 850 - 850 * 0.10
    else:
        meal_cost = 850

    if campus_hours >= 4:
```

```
        shuttle_cost = 250
    else:
        shuttle_cost = 0

    if event_registered == "yes":
        event_cost = 650
    else:
        event_cost = 0

    total_cost = meal_cost + shuttle_cost + event_cost
    print("Meal cost: USD", meal_cost)
    print("Shuttle cost: USD", shuttle_cost)
    print("Event cost: USD", event_cost)
    print("Total planned cost: USD", total_cost)

    # Part 3 - Budget check
    if total_cost <= budget:
        print("Your plan is within budget.")
    else:
        print("Your plan is over budget by USD", total_cost - budget)

    # Part 4 - Decision rule
    if total_cost <= budget and event_registered == "yes":
        print("Recommendation: Attend the event.")
    elif total_cost <= budget:
        print("Recommendation: Check event registration.")
    else:
        print("Recommendation: Review optional costs.")
```

```
Available budget (USD): 2000
Hours on campus: 5
Valid student card? (yes/no): yes
Registered for the event? (yes/no): yes
Meal cost: USD 765.0
Shuttle cost: USD 250
Event cost: USD 650
Total planned cost: USD 1665.0
Your plan is within budget.
Recommendation: Attend the event.
```

```
# Exercise 3, Test 2: within budget, not registered, no card
# Exercise 3, Part 1 - Input and validation
budget = float(input("Available budget (USD): "))
campus_hours = float(input("Hours on campus: "))
card_valid = input("Valid student card? (yes/no): ").lower()
event_registered = input("Registered for the event? (yes/no): ").lower()

if budget < 0 or campus_hours < 0:
    print("Error: budget and campus hours cannot be negative.")
else:
    # Part 2 - Calculate the plan
    if card_valid == "yes":
        meal_cost = 850 - 850 * 0.10
    else:
        meal_cost = 850

    if campus_hours >= 4:
        shuttle_cost = 250
    else:
        shuttle_cost = 0

    if event_registered == "yes":
        event_cost = 650
    else:
        event_cost = 0

    total_cost = meal_cost + shuttle_cost + event_cost
    print("Meal cost: USD", meal_cost)
    print("Shuttle cost: USD", shuttle_cost)
    print("Event cost: USD", event_cost)
    print("Total planned cost: USD", total_cost)

    # Part 3 - Budget check
    if total_cost <= budget:
        print("Your plan is within budget.")
    else:
        print("Your plan is over budget by USD", total_cost - budget)
```

```
    # Part 4 - Decision rule
    if total_cost <= budget and event_registered == "yes":
        print("Recommendation: Attend the event.")
    elif total_cost <= budget:
        print("Recommendation: Check event registration.")
    else:
        print("Recommendation: Review optional costs.")
```

```
Available budget (USD): 1500
Hours on campus: 3
Valid student card? (yes/no): no
Registered for the event? (yes/no): no
Meal cost: USD 850
Shuttle cost: USD 0
Event cost: USD 0
Total planned cost: USD 850
Your plan is within budget.
Recommendation: Check event registration.
```

```
# Exercise 3, Test 3: over budget, registered, valid card
# Exercise 3, Part 1 - Input and validation
budget = float(input("Available budget (USD): "))
campus_hours = float(input("Hours on campus: "))
card_valid = input("Valid student card? (yes/no): ").lower()
event_registered = input("Registered for the event? (yes/no): ").lower()

if budget < 0 or campus_hours < 0:
    print("Error: budget and campus hours cannot be negative.")
else:
    # Part 2 - Calculate the plan
    if card_valid == "yes":
        meal_cost = 850 - 850 * 0.10
    else:
        meal_cost = 850

    if campus_hours >= 4:
        shuttle_cost = 250
    else:
        shuttle_cost = 0

    if event_registered == "yes":
        event_cost = 650
    else:
        event_cost = 0

    total_cost = meal_cost + shuttle_cost + event_cost
    print("Meal cost: USD", meal_cost)
    print("Shuttle cost: USD", shuttle_cost)
    print("Event cost: USD", event_cost)
    print("Total planned cost: USD", total_cost)

    # Part 3 - Budget check
    if total_cost <= budget:
        print("Your plan is within budget.")
    else:
        print("Your plan is over budget by USD", total_cost - budget)

    # Part 4 - Decision rule
    if total_cost <= budget and event_registered == "yes":
        print("Recommendation: Attend the event.")
    elif total_cost <= budget:
        print("Recommendation: Check event registration.")
    else:
        print("Recommendation: Review optional costs.")
```

```
Available budget (USD): 1000
Hours on campus: 6
Valid student card? (yes/no): yes
Registered for the event? (yes/no): yes
Meal cost: USD 765.0
Shuttle cost: USD 250
Event cost: USD 650
Total planned cost: USD 1665.0
Your plan is over budget by USD 665.0
Recommendation: Review optional costs.
```

Testing different cases helped me check that each cost only applies under its own condition. Test 2 (3 hours, no card, not registered) confirmed the shuttle and event costs correctly drop to 0 and the meal stays at the full USD 850. Test 1 and Test 3 used a valid card and confirmed the 10% meal discount, and Test 3 confirmed that the over-budget amount and the "Review optional costs" recommendation appear. Without these tests, I could have missed logic errors such as charging for the shuttle when the student stays under 4 hours, using > instead of >= at the 4-hour boundary, or recommending the event to a student who is over budget.

```
Start coding or generate with AI.
```